

Epicode - CS0326IT

Build Week III

Esercizio 3: Navigare nel Filesystem Linux e Impostazioni dei Permessi

Gruppo: NetRunners

Parte 1: Esplorare i Filesystem in Linux

Passo 1 e 2 — Visualizzare i dispositivi a blocchi e i filesystem montati

Per prima cosa ho aperto il terminale sulla VM CyberOps Workstation e ho usato il comando `lsblk` per visualizzare tutti i dispositivi a blocchi (dischi) riconosciuti dal sistema:

```
[analyst@secOps ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   10G  0 disk
└─sda1       8:1    0   10G  0 part /
sdb          8:16   0    1G  0 disk
└─sdb1       8:17   0 1023M  0 part
sr0         11:0    1 1024M  0 rom

[analyst@secOps ~]$
```

Output del comando `lsblk`: due dischi (`sda` da 10GB e `sdb` da 1GB), ciascuno con una partizione.

L'output mostra che la VM ha due dischi installati: `sda` (10GB, con partizione `sda1` montata su `/`) e `sdb` (1GB, con partizione `sdb1` non ancora montata), oltre a `sr0` (un'unità CD/DVD virtuale).

In seguito ho usato il comando `mount` per vedere informazioni più dettagliate su tutti i filesystem attivi, e poi ho filtrato l'output con `grep` per isolare solo la riga relativa al filesystem radice: `mount | grep sda1`

```
[analyst@secOps ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   10G  0 disk
└─sda1       8:1    0   10G  0 part /
sdb          8:16   0    1G  0 disk
└─sdb1       8:17   0 1023M  0 part
sr0         11:0    1 1024M  0 rom

[analyst@secOps ~]$ mount
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
sys on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
dev on /dev type devtmpfs (rw,nosuid,relatime,size=995444k,nr_inodes=248861,mode=755,inode64)
run on /run type tmpfs (rw,nosuid,nodev,relatime,mode=755,inode64)
/dev/sda1 on / type ext4 (rw,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=41,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_ino=2046)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,nosuid,nodev,relatime,pagesize=2M)
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)
tracefs on /sys/kernel/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,nr_inodes=1048576,inode64)
tmpfs on /run/credentials/systemd-journald.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,relatime)
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /run/credentials/systemd-networkd.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
tmpfs on /run/user/1000 type tmpfs (rw,nosuid,nodev,relatime,size=200916k,nr_inodes=50229,mode=700,uid=1000,gid=1000,inode64)
[analyst@secOps ~]$ mount | grep sda1
/dev/sda1 on / type ext4 (rw,relatime)
[analyst@secOps ~]$
```

Output completo di `lsblk`, `mount` e `mount | grep sda1`.

Domanda: Qual è il significato dell'output? Dove sono fisicamente memorizzati i file elencati?

Risposta: L'output `"/dev/sda1 on / type ext4 (rw,relatime)"` indica che la partizione `/dev/sda1` (la prima partizione del disco sda) è montata sulla cartella radice `/`, usando il filesystem di tipo `ext4`, in modalità lettura-scrittura. Questo significa che tutti i file visibili nel filesystem radice (compresi il sistema operativo, i programmi e le configurazioni) sono fisicamente memorizzati sul disco sda, all'interno della partizione sda1.

Domanda: Perché `/dev/sdb1` non viene mostrato nell'output sopra?

Risposta: Perché `/dev/sdb1` esiste come dispositivo a blocchi (il sistema lo riconosce, come visto con `lsblk`), ma non è ancora stato montato. Il comando `mount` elenca solo i filesystem attualmente montati e accessibili: finché una partizione non viene collegata a una cartella con il comando `mount`, non compare in questo elenco, anche se il disco è fisicamente presente.

Passo 3 — Montare e smontare manualmente i filesystem

Ho verificato che la cartella `second_drive` (dentro la mia home) fosse vuota, poi ho montato la partizione `/dev/sdb1` su di essa, ho controllato il contenuto, e infine ho smontato tutto correttamente:

```
[analyst@secOps ~]$ ls -l second_drive/
ls: cannot access 'second_drive/': No such file or directory
[analyst@secOps ~]$ cd ~
[analyst@secOps ~]$ ls -l second_drive/
total 0
[analyst@secOps ~]$ sudo mount /dev/sdb1 ~/second_drive/
[sudo] password for analyst:
[analyst@secOps ~]$ ls -l second_drive/
total 20
drwx----- 2 root    root    16384 Mar 26  2018 lost+found
-rw-r--r--  1 analyst analyst  183 Mar 26  2018 myFile.txt
[analyst@secOps ~]$ mount | grep /dev/sd
/dev/sda1 on / type ext4 (rw,relatime)
/dev/sdb1 on /home/analyst/second_drive type ext4 (rw,relatime)
[analyst@secOps ~]$ sudo umount /dev/sdb1
[analyst@secOps ~]$ ls -l second_drive/
total 0
[analyst@secOps ~]$
```

Sequenza completa: verifica cartella vuota, montaggio di `/dev/sdb1`, contenuto visibile, smontaggio e cartella tornata vuota.

Domanda: Perché la directory non è più vuota? Dove sono fisicamente memorizzati i file elencati?

Risposta: Dopo il comando `"sudo mount /dev/sdb1 ~/second_drive"`, la cartella `second_drive` smette di essere una semplice cartella vuota locale e diventa il punto di montaggio della partizione `/dev/sdb1`. Da quel momento, tutto ciò che è fisicamente scritto sul disco sdb (in questo caso il file `myFile.txt` e la cartella `lost+found`) diventa visibile e accessibile attraverso quella cartella. I file, quindi, sono fisicamente memorizzati sul disco sdb (partizione sdb1), non su sda dove si trova il resto del sistema.

Dopo aver eseguito `"sudo umount /dev/sdb1"` trovandomi nella mia home (fuori dalla cartella `second_drive`, per evitare l'errore `"device is busy"`), la cartella `second_drive` è tornata vuota

(total 0): il disco fisico è stato scollegato dal punto di montaggio, quindi i file non sono più visibili da lì, anche se continuano a esistere fisicamente sul disco sdb1.

Parte 2: Permessi dei File

Passo 1 — Visualizzare e modificare i permessi dei file

Mi sono spostato nella cartella degli script e ho elencato i file con i dettagli dei permessi:

Domanda: Considera il file *cyops.mn* come esempio. Chi è il proprietario del file? E il gruppo? I permessi per *cyops.mn* sono *-rw-r--r--*. Cosa significa?

Risposta: Il proprietario del file *cyops.mn* è l'utente *analyst*, e anche il gruppo è *analyst*. I permessi *-rw-r--r--* si leggono così: il primo carattere "-" indica che è un file regolare (non una directory). I successivi 9 caratteri sono divisi in 3 gruppi da 3: "rw-" per il proprietario (può leggere e scrivere, ma non eseguire), "r--" per il gruppo (può solo leggere) e "r--" per tutti gli altri utenti (possono solo leggere).

Ho poi provato a creare un nuovo file vuoto nella cartella */mnt* con il comando *touch*, ottenendo un errore di permesso negato:

```
[analyst@secOps ~]$ cd lab.support.files/scripts/
[analyst@secOps scripts]$ ls -l
total 68
-rwxr-xr-x 1 analyst analyst 952 Mar 21 2018 configure_as_dhcp.sh
-rwxr-xr-x 1 analyst analyst 1153 Mar 21 2018 configure_as_static.sh
-rwxr-xr-x 1 analyst analyst 4053 Jun 18 2025 cyberops_extended_topo_no_fw.py
-rwxr-xr-x 1 analyst analyst 5016 Jun 18 2025 cyberops_extended_topo.py
-rwxr-xr-x 1 analyst analyst 4189 Jun 18 2025 cyberops_topo.py
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rwxr-xr-x 1 analyst analyst 458 Mar 21 2018 fw_rules
-rwxr-xr-x 1 analyst analyst 70 Mar 21 2018 mal_server_start.sh
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 net_configuration_files
-rwxr-xr-x 1 analyst analyst 65 Mar 21 2018 reg_server_start.sh
-rwxr-xr-x 1 analyst analyst 189 Mar 21 2018 start_ELK.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 18 2025 start_miniedit.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 19 2025 start_pox.sh
-rwxr-xr-x 1 analyst analyst 117 Jun 19 2025 start_snort.sh
-rwxr-xr-x 1 analyst analyst 61 Mar 21 2018 start_tftpd.sh
[analyst@secOps scripts]$ touch /mnt/myNewFile.txt
touch: cannot touch '/mnt/myNewFile.txt': Permission denied
[analyst@secOps scripts]$ ls -ld /mnt
drwxr-xr-x 2 root root 4096 Jan 5 2018 /mnt
[analyst@secOps scripts]$ sudo touch /mnt/myNewFile.txt
[sudo] password for analyst:
[analyst@secOps scripts]$ sudo mount /dev/sdb1 ~/second_drive/
[analyst@secOps scripts]$ cd ~/second_drive
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar 26 2018 lost+found
-rw-r--r-- 1 analyst analyst 183 Mar 26 2018 myFile.txt
[analyst@secOps second_drive]$
```

touch fallisce con *Permission denied*; verifica dei permessi con *ls -ld /mnt*; risoluzione con *sudo touch* e successivo montaggio di *second_drive*.

Domanda: Perché il file non è stato creato? Elenca i permessi, la proprietà e il contenuto della directory /mnt e spiega cosa è successo. Con l'aggiunta dell'opzione -d, elenca i permessi della directory genitore.

Risposta: Il file non è stato creato perché non avevo i permessi di scrittura sulla cartella /mnt. Con "ls -ld /mnt" ho verificato i permessi della cartella stessa (non il suo contenuto): drwxr-xr-x, proprietario root, gruppo root. Poiché il mio utente analyst non è né il proprietario né appartiene al gruppo root, rientro nella categoria "altri", che ha solo i permessi r-x (lettura ed accesso alla cartella, ma non scrittura). Per questo motivo il comando touch, che ha bisogno del permesso di scrittura sulla cartella per poter creare un nuovo file al suo interno, fallisce con "Permission denied".

Elencando il contenuto della cartella /mnt con "ls -l /mnt" (senza l'opzione -d), il risultato mostra il file myNewFile.txt creato successivamente con sudo.

Il file myNewFile.txt è di proprietà di root e ha permessi -rw-r--r--, coerentemente con il fatto che è stato creato usando sudo, cioè con i privilegi di amministratore.

```
[analyst@secOps lab.support.files]$ ls -l /mnt
total 0
-rw-r--r-- 1 root root 0 Sep  7 07:11 myNewFile.txt
[analyst@secOps lab.support.files]$
```

Contenuto della cartella /mnt: il file myNewFile.txt, proprietario root, creato con sudo touch.

Domanda: Cosa si può fare affinché il comando touch mostrato sopra abbia successo?

Risposta: Ci sono principalmente due soluzioni. La prima, quella che ho usato, è eseguire il comando con i privilegi di amministratore tramite sudo (sudo touch /mnt/myNewFile.txt), che permette di aggirare temporaneamente la restrizione di permesso. La seconda soluzione, alternativa, sarebbe cambiare i permessi della cartella /mnt con chmod (ad esempio dando il permesso di scrittura anche agli altri utenti), operazione che però richiederebbe comunque i privilegi di root, dato che solo il proprietario o l'amministratore possono modificare i permessi di un file o cartella altrui.

Ho poi rimontato il disco sdb1 su second_drive e ho verificato i permessi del file myFile.txt al suo interno:

Domanda: Quali sono i permessi del file myFile.txt?

Risposta: I permessi iniziali di myFile.txt erano -rw-r--r-- (proprietario: lettura e scrittura; gruppo e altri: solo lettura).

Ho quindi cambiato i permessi con il comando chmod, cambiato temporaneamente il proprietario con chown per fare un confronto, e infine verificato la scrittura sul file:

```
[analyst@secOps second_drive]$ sudo chmod 665 myFile.txt
[analyst@secOps second_drive]$ sudo chown root myFile.txt
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar 26 2018 lost+found
-rw-rw-r-x 1 root analyst 183 Mar 26 2018 myFile.txt
[analyst@secOps second_drive]$ sudo chown analyst myFile.txt
[analyst@secOps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar 26 2018 lost+found
-rw-rw-r-x 1 analyst analyst 183 Mar 26 2018 myFile.txt
[analyst@secOps second_drive]$ echo "test" >> myFile.txt
[analyst@secOps second_drive]$ cat myFile.txt
This is a file stored in the /dev/sdb1 disk.
Notice that even though this file has been sitting in this disk for a while, it couldn't be accessed until the disk was properly mounted.
test
[analyst@secOps second_drive]$
```

chmod 665 applicato; chown root e successivamente chown analyst; scrittura riuscita nel file senza bisogno di sudo.

Domanda: *I permessi sono cambiati? Quali sono i permessi di myFile.txt?*

Risposta: Sì, i permessi sono cambiati: dopo "chmod 665 myFile.txt" i nuovi permessi sono -rw-rw-r-x. Il numero 665 in ottale si scompone così: 6 = rw- (4+2) per il proprietario, 6 = rw- (4+2) per il gruppo, 5 = r-x (4+1) per gli altri. Quindi ora sia il proprietario che il gruppo possono leggere e scrivere, mentre gli altri utenti possono leggere ed "eseguire" (bit che su un file di testo non ha un effetto pratico, ma viene comunque impostato).

Domanda: *Quale comando cambierebbe i permessi di myFile.txt a rwxrwxrwx, garantendo a qualsiasi utente nel sistema pieno accesso al file?*

Risposta: Il comando sarebbe "sudo chmod 777 myFile.txt", perché 7 in ottale corrisponde a 4+2+1, cioè rwx (lettura, scrittura ed esecuzione) per ciascuna delle tre categorie: proprietario, gruppo e altri.

Ho poi provato a cambiare proprietario, prima portandolo a root e poi di nuovo ad analyst, per osservare concretamente l'effetto del comando chown (nella mia VM il file era già di proprietà di analyst fin dall'inizio)

Infine ho aggiunto del testo al file con echo e verificato il contenuto con cat

Domanda: *L'operazione è riuscita? Spiega.*

Risposta: Sì, l'operazione di scrittura con "echo "test" >> myFile.txt" è riuscita senza bisogno di sudo. Questo perché, dopo il comando chown, l'utente analyst è tornato ad essere il proprietario del file, e grazie al chmod 665 precedente il proprietario ha il permesso di scrittura (rw-). Di conseguenza il sistema operativo mi ha autorizzato a scrivere direttamente nel file, senza dover elevare i privilegi con sudo. Nel file era inoltre già presente un messaggio informativo che ricordava come, anche se il file esiste fisicamente sul disco sdb1 da tempo, non era accessibile finché il disco non veniva montato correttamente.

Passo 2 — Directory e permessi

Sono tornato nella cartella *lab.support.files* e ho elencato il contenuto con i dettagli, includendo anche un confronto tra una directory e un file eseguibile, e infine ho osservato i tipi di file speciali nella cartella */dev*:

```

[analyst@secOps second_drive]$ cd ~/lab.support.files/
[analyst@secOps lab.support.files]$ ls -l
total 580
-rw-r--r-- 1 analyst analyst 649 Mar 21 2018 apache_in_epoch.log
-rw-r--r-- 1 analyst analyst 126 Mar 21 2018 applicationX_in_epoch.log
drwxr-xr-x 4 analyst analyst 4096 Mar 21 2018 attack_scripts
-rw-r--r-- 1 analyst analyst 102 Mar 21 2018 confidential.txt
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rw-r--r-- 1 analyst analyst 75 Mar 21 2018 elk_services
-rw-r--r-- 1 analyst analyst 373 Mar 21 2018 h2_dropbear.banner
drwxr-xr-x 2 analyst analyst 4096 Apr 2 2018 instructor
-rw-r--r-- 1 analyst analyst 255 Mar 21 2018 letter_to_grandma.txt
-rw-r--r-- 1 analyst analyst 24464 Mar 21 2018 logstash-tutorial.log
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 malware
-rwxr-xr-x 1 analyst analyst 172 Mar 21 2018 mininet_services
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 openssl_lab
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 pcaps
drwxr-xr-x 6 analyst analyst 4096 Aug 15 2022 pox
-rw-r--r-- 1 analyst analyst 473363 Mar 21 2018 sample.img
-rw-r--r-- 1 analyst analyst 65 Mar 21 2018 sample.img_SHA256.sig
drwxr-xr-x 3 analyst analyst 4096 Jun 18 2025 scripts
-rw-r--r-- 1 analyst analyst 25553 Mar 21 2018 SQL_Lab.pcap
[analyst@secOps lab.support.files]$ ls -l /dev/
total 0
crw-r--r-- 1 root root 10, 235 Sep 7 05:28 autofs
drwxr-xr-x 2 root root 140 Sep 7 05:28 block
drwxr-xr-x 2 root root 100 Sep 7 05:28 bsg
crw-rw---- 1 root disk 10, 234 Sep 7 05:28 btrfs-control
drwxr-xr-x 3 root root 60 Sep 7 05:28 bus
lrwxrwxrwx 1 root root 3 Sep 7 05:28 cdrom -> sr0
drwxr-xr-x 2 root root 3780 Sep 7 05:28 char
crw----- 1 root root 5, 1 Sep 7 05:28 console
lrwxrwxrwx 1 root root 11 Sep 7 05:28 core -> /proc/kcore
drwxr-xr-x 4 root root 80 Sep 7 05:28 cpu
crw----- 1 root root 10, 124 Sep 7 05:28 cpu_dma_latency
crw----- 1 root root 10, 203 Sep 7 05:28 cuse
drwxr-xr-x 7 root root 140 Sep 7 05:28 disk
drwxr-xr-x 2 root root 60 Sep 7 05:28 dma_heap
drwxr-xr-x 3 root root 100 Sep 7 05:28 dri
crw-rw---- 1 root video 29, 0 Sep 7 05:28 fb0
lrwxrwxrwx 1 root root 13 Sep 7 05:28 fd -> /proc/self/fd
crw-rw-rw- 1 root root 1, 7 Sep 7 05:28 full
crw-rw-rw- 1 root root 10, 229 Sep 7 05:28 fuse
crw----- 1 root root 244, 0 Sep 7 05:28 hidraw0
crw----- 1 root root 10, 228 Sep 7 05:28 hpet
drwxr-xr-x 2 root root 0 Sep 7 05:28 hugepages
crw----- 1 root root 10, 183 Sep 7 05:28 hwrng
drwxr-xr-x 4 root root 360 Sep 7 05:28 input
crw-r--r-- 1 root root 1, 11 Sep 7 05:28 kmsg
lrwxrwxrwx 1 root root 28 Sep 7 05:28 log -> /run/systemd/journal/dev-log
crw-rw---- 1 root disk 10, 237 Sep 7 05:28 loop-control

```

Elenco di lab.support.files (con malware e mininet_services evidenziati) e output di ls -l /dev/.

Confrontando le due righe:

```

drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 malware
-rwxr-xr-x 1 analyst analyst 172 Mar 21 2018 mininet_services

```


Domanda: Qual è la differenza tra la parte iniziale della riga di malware e la riga di mininet_services?

Risposta: La differenza principale è il primo carattere di ogni riga: "d" per malware indica che si tratta di una directory (cartella), mentre "-" per mininet_services indica che si tratta di un file regolare. Inoltre il significato di "x" cambia in base al tipo: su un file come mininet_services, x significa che il file può essere eseguito come programma o script; su una directory come malware, invece, x significa che un utente autorizzato può "entrarci dentro" con il comando cd, non che la cartella si possa "eseguire". I comandi chmod e chown funzionano comunque allo stesso modo sia sui file che sulle directory.

Parte 3: Link Simbolici e Altri Tipi di File Speciali

Passo 1 — Esaminare i tipi di file

Ho analizzato l'output di "ls -l /dev/" (visibile nello screenshot sopra) per individuare i diversi tipi di file speciali. In particolare ho notato:

- File a blocco (b): ad esempio sda, sdb — rappresentano l'accesso fisico agli hard disk
- File a carattere (c): ad esempio random, tty — forniscono un flusso seriale di input/output
- Link simbolici (l): ad esempio cdrom -> sr0, core -> /proc/kcore — collegamenti che puntano ad altri file

Passo 2 e 3 — Creare link simbolici e hard link

Ho creato due file di testo, poi un link simbolico verso il primo e un hard link verso il secondo:

```
[analyst@secOps lab.support.files]$ echo "symbolic" > file1.txt
[analyst@secOps lab.support.files]$ cat file1.txt
symbolic
[analyst@secOps lab.support.files]$ echo "hard" > file2.txt
[analyst@secOps lab.support.files]$ cat file2.txt
hard
[analyst@secOps lab.support.files]$ ln -s file1.txt file1symbolic
[analyst@secOps lab.support.files]$ ln file2.txt file2hard
[analyst@secOps lab.support.files]$ ls -l
total 592
-rw-r--r-- 1 analyst analyst 649 Mar 21 2018 apache_in_epoch.log
-rw-r--r-- 1 analyst analyst 126 Mar 21 2018 applicationX_in_epoch.log
drwxr-xr-x 4 analyst analyst 4096 Mar 21 2018 attack_scripts
-rw-r--r-- 1 analyst analyst 102 Mar 21 2018 confidential.txt
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rw-r--r-- 1 analyst analyst 75 Mar 21 2018 elk_services
lrwxrwxrwx 1 analyst analyst 9 Sep 7 08:24 file1symbolic -> file1.txt
-rw-r--r-- 1 analyst analyst 9 Sep 7 08:23 file1.txt
-rw-r--r-- 2 analyst analyst 5 Sep 7 08:23 file2hard
-rw-r--r-- 2 analyst analyst 5 Sep 7 08:23 file2.txt
-rw-r--r-- 1 analyst analyst 373 Mar 21 2018 h2_dropbear.banner
drwxr-xr-x 2 analyst analyst 4096 Apr 2 2018 instructor
-rw-r--r-- 1 analyst analyst 255 Mar 21 2018 letter_to_grandma.txt
-rw-r--r-- 1 analyst analyst 24464 Mar 21 2018 logstash-tutorial.log
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 malware
-rwxr-xr-x 1 analyst analyst 172 Mar 21 2018 mininet_services
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 openssl_lab
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 pcaps
drwxr-xr-x 6 analyst analyst 4096 Aug 15 2022 pox
-rw-r--r-- 1 analyst analyst 473363 Mar 21 2018 sample.img
-rw-r--r-- 1 analyst analyst 65 Mar 21 2018 sample.img_SHA256.sig
drwxr-xr-x 3 analyst analyst 4096 Jun 18 2025 scripts
-rw-r--r-- 1 analyst analyst 25553 Mar 21 2018 SQL_Lab.pcap
[analyst@secOps lab.support.files]$
```

Creazione dei file, del link simbolico (file1symbolic -> file1.txt) e dell'hard link (file2hard).

Dall'output di `ls -l` si nota che `file1symbolic` inizia con la lettera "l" e mostra una freccia (->) verso `file1.txt`: è un link simbolico. Il file `file2hard`, invece, appare come un normale file regolare (-), ma nella quinta colonna (dove di solito compare 1) mostra il numero 2, sia per `file2hard` che per `file2.txt`: questo indica che sono due nomi diversi collegati allo stesso identico dato fisico sul disco (stesso inode).

Ho poi rinominato i due file originali per osservare l'effetto sui rispettivi collegamenti:

```
[analyst@secOps lab.support.files]$ echo "symbolic" > file1.txt
[analyst@secOps lab.support.files]$ cat file1.txt
symbolic
[analyst@secOps lab.support.files]$ echo "hard" > file2.txt
[analyst@secOps lab.support.files]$ cat file2.txt
hard
[analyst@secOps lab.support.files]$ ln -s file1.txt file1symbolic
[analyst@secOps lab.support.files]$ ln file2.txt file2hard
[analyst@secOps lab.support.files]$ ls -l
total 592
-rw-r--r-- 1 analyst analyst 649 Mar 21 2018 apache_in_epoch.log
-rw-r--r-- 1 analyst analyst 126 Mar 21 2018 applicationX_in_epoch.log
drwxr-xr-x 4 analyst analyst 4096 Mar 21 2018 attack_scripts
-rw-r--r-- 1 analyst analyst 102 Mar 21 2018 confidential.txt
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rw-r--r-- 1 analyst analyst 75 Mar 21 2018 elk_services
lrwxrwxrwx 1 analyst analyst 9 Sep 7 08:24 file1symbolic -> file1.txt
-rw-r--r-- 1 analyst analyst 9 Sep 7 08:23 file1.txt
-rw-r--r-- 2 analyst analyst 5 Sep 7 08:23 file2hard
-rw-r--r-- 2 analyst analyst 5 Sep 7 08:23 file2.txt
-rw-r--r-- 1 analyst analyst 373 Mar 21 2018 h2_dropbear.banner
drwxr-xr-x 2 analyst analyst 4096 Apr 2 2018 instructor
-rw-r--r-- 1 analyst analyst 255 Mar 21 2018 letter_to_grandma.txt
-rw-r--r-- 1 analyst analyst 24464 Mar 21 2018 logstash-tutorial.log
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 malware
-rwxr-xr-x 1 analyst analyst 172 Mar 21 2018 mininet_services
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 openssl_lab
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 pcaps
drwxr-xr-x 6 analyst analyst 4096 Aug 15 2022 pox
-rw-r--r-- 1 analyst analyst 473363 Mar 21 2018 sample.img
-rw-r--r-- 1 analyst analyst 65 Mar 21 2018 sample.img_SHA256.sig
drwxr-xr-x 3 analyst analyst 4096 Jun 18 2025 scripts
-rw-r--r-- 1 analyst analyst 25553 Mar 21 2018 SQL_Lab.pcap
[analyst@secOps lab.support.files]$ mv file1.txt file1new.txt
[analyst@secOps lab.support.files]$ mv file2.txt file2new.txt
[analyst@secOps lab.support.files]$ cat file1symbolic
cat: file1symbolic: No such file or directory
[analyst@secOps lab.support.files]$ cat file2hard
hard
[analyst@secOps lab.support.files]$
```

Dopo la rinomina: `file1symbolic` risulta un link rotto (No such file or directory), mentre `file2hard` funziona ancora correttamente.

Il risultato mostra che `cat file1symbolic` dà errore "No such file or directory", perché il link simbolico punta al nome `file1.txt`, che non esiste più (è stato rinominato in `file1new.txt`): si tratta quindi di un link simbolico rotto. Al contrario, `cat file2hard` restituisce ancora

correttamente il testo "hard", perché l'hard link non dipende dal nome del file ma punta direttamente al dato fisico (inode) sul disco, che resta raggiungibile anche dopo la rinomina.

Domanda: Cosa pensi succederebbe a file2hard se aprissi un editor di testo e cambiassi il testo in file2new.txt?

Risposta: Il contenuto di file2hard cambierebbe di conseguenza, mostrando il nuovo testo. Questo perché file2hard e file2new.txt non sono due file distinti con contenuti separati, ma due nomi diversi che puntano allo stesso identico inode, cioè agli stessi dati fisici sul disco. Modificando il contenuto tramite uno dei due nomi, la modifica sarà visibile leggendo il file anche con l'altro nome, perché in realtà si tratta dello stesso file.